Spoil-Less

Technical Design Document

Date Updated: 2026/03/18
Version 1.0
Team: Sonny Chen, Rivka Steinberg, Benny Guo

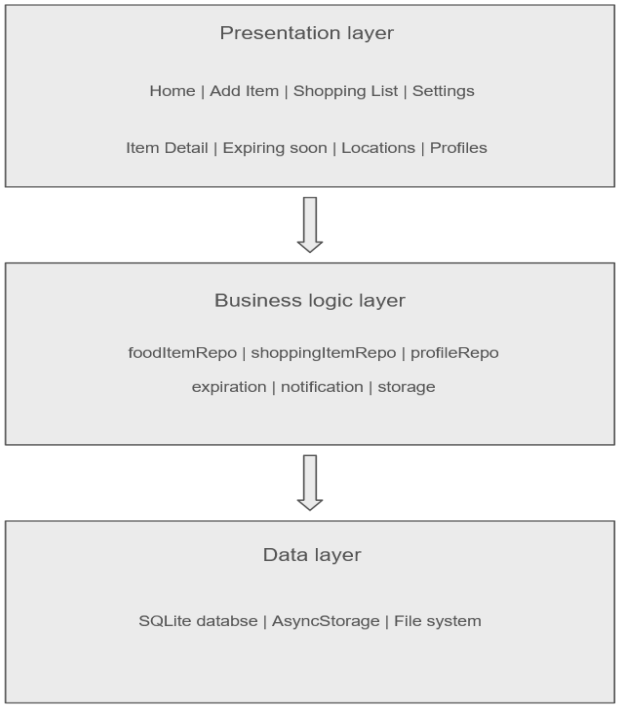
Context

This document describes the technical design for Spoil-Less, a mobile application that helps users track their groceries and their expiration dates in an effort to reduce food waste. The requirements document can be found here: Spoil-Less Requirements Doc.

The primary technical challenge for this project is building an app that is responsive, completely offline, persists data locally on a single device, supports multiple user profiles, and is able to deliver mobile notifications without depending on any sort of network. To address these problems, the application will make use of a local SQLite database, a notification scheduler that does not require network availability, and clear separation between the UI layer, logic layer, and data layer.

Architecture Design

Architecture Diagram



Key Architectural Decisions

1. Layered Architecture

The application is divided into three layers: Presentation, Business Logic, and Data. Each layer only has one core responsibility and communicates only with layers adjacent to it.

2. Single Local Database as the source of truth

All the data that needs to persist, such as: tracked items, profiles, storage locations, shopping lists, and notification preferences, would reside in the SQLite database on the device. This eliminates any concerns about synchronization and network availability, ensuring the app can be used offline at any time.

3. Data Access through Repositories

The business logic layer accesses data through repository modules (like foodItemRepository or profileRepository). This allows clean encapsulation and isolation of all database queries and responsibilities. The repository's job is to fetch, insert, update, and delete data, with the UI's job

simply being to call the correct one. There is no direct access from the Presentation layer to the Data layer.

4. Notification Scheduling handled locally
The notification system uses the device's local notification API (through expo notifications) to schedule alerts depending on each item's expiration date. The notifications would be scheduled at the creation of an item or when it's edited, and are cancelled when items are discarded or used. This allows operation without any network connection, allowing full offline usability.

5. Profile Isolated Data
Each user profile maintains its own set of tracked items, storage locations, shopping lists, etc. The active user would be tracked with a unique ID to filter all reads and writes. This way, profiles stay separated even though the same underlying database and device is being used.

Technology Stack

Layer	Choice
Mobile Framework	React Native (Expo)
Language	JavaScript
Local Database	SQLite
State Management	React Context
Local Notifications	Expo notifications
Image Storage	The device file system
Navigation	React Navigation
Preferences Storage	AsyncStorage

System Components

Data Layer Components

DatabaseManager:
The DatabaseManager is the setup and access point for the app's database. It creates the SQLite database when the application launches, and creates all the required tables if they don't already exist. It also provides the database access point that the repository modules in the

business logic layer can use to query the tables. It also handles the creation of a default profile as well as populating the default expiry lookup table.

Business Logic Components

profileRepository

Manages all the operations relating to the user's profiles. Holds the current active profile ID which all other repositories use to correctly scope their queries. Handles creating new profiles, switching profiles, renaming profiles, and deleting profiles. If a profile is ever deleted, the repository would also consequently delete all associated data like tracked items, storage locations, shopping list entries, and scheduled notifications.

foodItemRepository

Handles the CRUD operations for all the tracked food entries. Adds new items and assigns default expiration if necessary (via expirationService), edits item details, marks items as used or discarded, and deletes items. When an item is created or if its expiration date is modified, this repository will work with notificationScheduler to schedule/update its expiration date. It also handles filtering and sorting functions, whether it be by category, location, or expiration date.

storageLocationRepository

Manages the creation, renaming, and deleting of the user made storage locations. Also ensures that the user can not delete a storage location while it still contains active tracked items. If the user attempts to do so, the repository would return a flag that indicates confirmation is required. The presentation layer would then display the appropriate screen and dialogue.

shoppingListRepository

Manages the user's shopping list. Its responsibilities would include adding items, removing items, and transitioning items to active tracking. When this happens, the repository will delegate the item creation to foodItemRepository. It will also perform a name match against all the active tracked items whenever a new shopping list entry is created, which would return a flag if a duplicate is detected so the UI can display a notice to let the user know they already have the item at home.

notificationScheduler

Responsible for scheduling and canceling notifications through expo-notifications. When a food item is added, the scheduler figures out the appropriate notification times and registers it with the device. When an item is removed or marked as used/discarded, the corresponding notifications are cancelled. The scheduler also makes sure to respect the user's notification preference, and only works if they opted in.

expirationService

Evaluates the expiration status for entries. Determines whether an item is "expiring soon" (this expiry window would be configurable, but defaults to 3 days), or if it has expired. This component would also manage the default expiration date logic for when a user does not provide an expiration date. expirationService would in this case look up the item name in a built in the default expiry lookup table. If no match is found, it would fall back to the 3 week default as mentioned in FR1.7 in the Requirements Doc.

Presentation Layer Components

AppState

The React Context provider. It holds the current active profile's ID and provides shared state and functions through context. All child screens like the home screen or shopping list would read from this context to decide which profile's data to display. AppState is instantiated once, and all screens are wrapped by it.

Screen Components

Each major screen featured in the app is its own individual React Native component, and calls the appropriate functions in the repositories and renders the user interface. The major screens include the home/dashboard screen, add item, item details, storage locations, shopping list, expiring soon, notification settings, and profile management. Navigation between all these different screens is handled by React Navigation.

Data Design

These would map to SQLite tables in the local database.

```
ItemStatus: "active" | "used" | "discarded"

profiles
  id                    TEXT PRIMARY KEY
  name                  TEXT NOT NULL
  createdAt             TEXT NOT NULL

```

```
storage_locations
  id                    TEXT PRIMARY KEY
  profileId             TEXT NOT NULL
  name                  TEXT NOT NULL

categories
  id                    TEXT PRIMARY KEY
  profileId             TEXT NOT NULL
  name                  TEXT NOT NULL

food_items
  id                    TEXT PRIMARY KEY
  profileId             TEXT NOT NULL
  name                  TEXT NOT NULL
  expirationDate        TEXT NOT NULL
  storageLocationId     TEXT NOT NULL
  categoryId            TEXT
  note                  TEXT
  photoUri              TEXT
  status                TEXT NOT NULL
  usedDefaultExpiry     INTEGER NOT NULL
  createdAt             TEXT NOT NULL

shopping_list_items
  id                    TEXT PRIMARY KEY
  profileId             TEXT NOT NULL
  name                  TEXT NOT NULL
  createdAt             TEXT NOT NULL

notification_preferences
  profileId             TEXT PRIMARY KEY
  enabled               INTEGER NOT NULL
  daysBefore            INTEGER NOT NULL

default_expiry_lookup
  itemName              TEXT PRIMARY KEY
  defaultDays           INTEGER NOT NULL
```

Key Data Decisions

1. Profile Scoped

Every piece of user data is tagged with the profile it belongs to using profileId. This allows all profiles to share one SQLite database by limiting the scope of a query to the appropriate profile ID. This also means that if a profile is deleted, the database can also delete all related rows to that profile.

2. Item Status as a field

Instead of maintaining a separate table for the status of a food, items instead have a status field. This simplifies the database and allows the application to filter active items, while also being able to easily access archived items for historical information/analytics.

3. Storing photos using file paths

Instead of storing user photos inside the SQLite database itself, the photos will be normal files on the phone, and the database would only store the path to where each photo is located through expo filesystem. This makes sure the database isn't bloated and the application/queries remain as fast as possible.

4. Built in default expiry lookup table

This provides better expiration accuracy over a pure 3-week default expiration timer. The table would map common household perishables to their average shelf life. The table is populated by the DatabaseManager on launch and is consulted by the expirationService if a user selects an item from the predefined list without manually entering an expiration date. If an item is not in this table, the system falls back to the default 3-weeks.

Detailed Design

Food Item Lifecycle

Adding a new item:

1. User goes to the Add Item screen.
2. User either selects or manually enter their desired entry.
3. If the item was selected from the predefined list, the expirationService looks up the default shelf life from the default expiry lookup table and prefills the expiration data field.
4. The user can either accept the prefilled date, or manually enter their own, or if no predefined match exists they can just default to the generic 3 week option via a toggle.
5. User selects a storage location for their entry and optionally could assign a category, note, and photo.
6. foodItemRepository makes sure the item has a name and an expiration date, then inserts it into the food_items table.

7. If the user has notifications enabled for the active profile, the notificationScheduler schedules the configured number of days before expiration.

Editing an item:

1. User selects an existing entry and taps edit.
2. User modifies a field (name, expiration date, storage location, category, note, photo).
3. The foodItemRepository updates the database.
4. If the expiration field is changed and notifications are enabled, the notificationScheduler schedules a new date.

Marking as Used or Discarded

1. User marks an item as “used” or “discarded” from the item detail screen.
2. The foodItemRepository updates the item’s status field.
3. The item is removed from the active tracking view.
4. The notificationScheduler cancels any pending notifications for that item.

Deleting an Item

1. User chooses to delete a tracked item.
2. The foodItemRepository removes the record from the database.
3. If a photo was attached, the file is deleted from the device file system.
4. The notificationScheduler cancels any pending notifications for that item.

Shopping List to Tracking Flow

When a user moves a purchased item from the shopping list to active tracking:

1. User selects an item on the shopping list and taps “move to tracking.”
2. The system moves to a variant of the regular add item screen with the item name already prefilled with whatever it’s called in the shopping list. Then, it prompts the user for expiration date, storage location, and optional fields like a photo or note.
3. After completing and confirming the entry, the shoppingListRepository deletes the shopping list entry and foodItemRepository creates the new tracked item.
4. If notifications are enabled, the notificationScheduler schedules the appropriate notification.

Storage Location Deletion Flow

When a user tries to delete a storage location:

1. The storageLocationRepository checks whether it contains any active tracked items in the location.
2. If the location is empty and there are no tracked items, it is deleted.

3. If the location contains active items, the repository returns a flag and the UI displays a popup to notify the user that there are still items in the location. The deletion is blocked until the user moves or removes the items from that location first.

Notification Scheduling Logic

Notifications are managed in this way:

- When a food item is created with an expiration date, and the user has notifications enabled, a local notification is scheduled at 9:00 AM local time. The date would be calculated with expirationDate - daysBefore. A second notification is scheduled on the actual expiration date itself to let the user know the item is spoiled.
- When an entry's expiration date is edited, the preexisting notifications are cancelled and new ones are scheduled.
- When a food item is deleted, used, or discarded, all notifications are cancelled.
- When a user toggles notifications off, all notifications for the profile are cancelled. If notifications are turned back on, notifications are rescheduled for all active items that haven't expired during that window.
- The system checks expiration dates using the device's local date and time.

Duplicate Item Handling

The system will allow users to add multiple entries of the same item name. This is because a user may purchase or have multiple of the same product with different expiration dates. They may also store identical items in different storage locations, like different fridges, cabinets, etc. Each entry would be treated as a distinct item with it's own fields.

The shopping list, although it checks for duplicates by seeing whether a user is adding an item to the shopping list that already exists in active tracking, is only informational. It displays to the user that they already have this item at home, but does not block them from adding it to the shopping list if they wish to buy another one anyway.

Profile Management Flow

The system supports multiple (local) user profiles, each with completely isolated data:

- On first launch, the DatabaseManager creates a default profile "My Profile."
- Users can create additional profiles from the Profile management screen. Each profile starts with no tracked items, no storage locations, and an empty shopping list.
- Switching profiles will update the active profile ID in asyncstorage and also triggers the system to refresh and reload all of the scoped data to that profile they switched to via AppStateProvider.

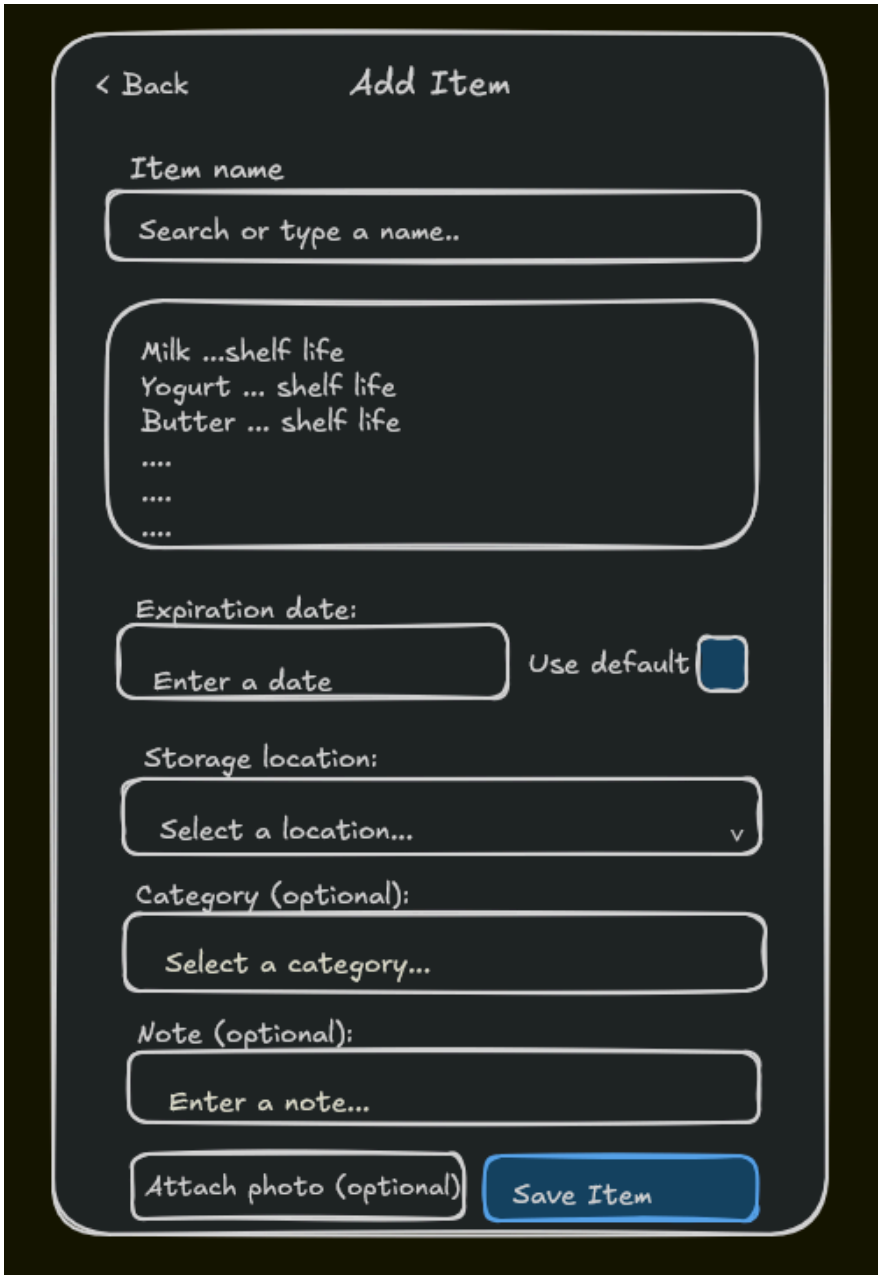
- Deleting a profile leads to a cascading effect where all associated data, such as the food items, storage locations, shopping list entries, notification preferences, and photos, are deleted as well. The system will ask for confirmation first, and will also prevent the user from deleting a profile if it is the last one remaining.

User Experience

Wireframes

Home / Dashboard:

Add Item Screen:



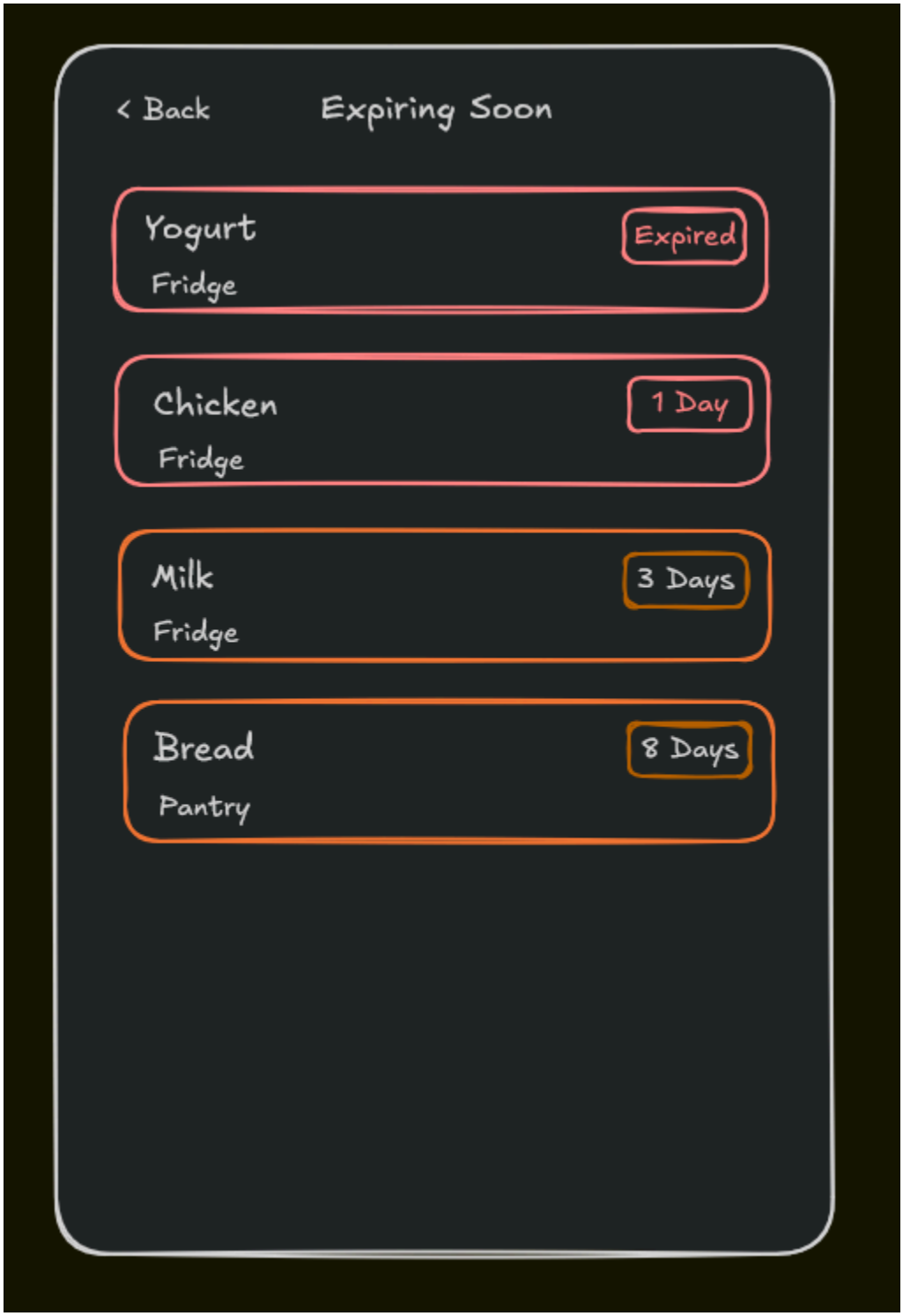
Item Detail:

< Back
Item Detail
Edit
Milk
1 Day
Expires: Mar 19, 2026
Added: Mar 5, 2026
Storage location
Fridge
Category
Dairy
Note
2% fat, organic, use remainder for cupcakes
Photo:
[image]
Mark as used
Discarded
Delete Item

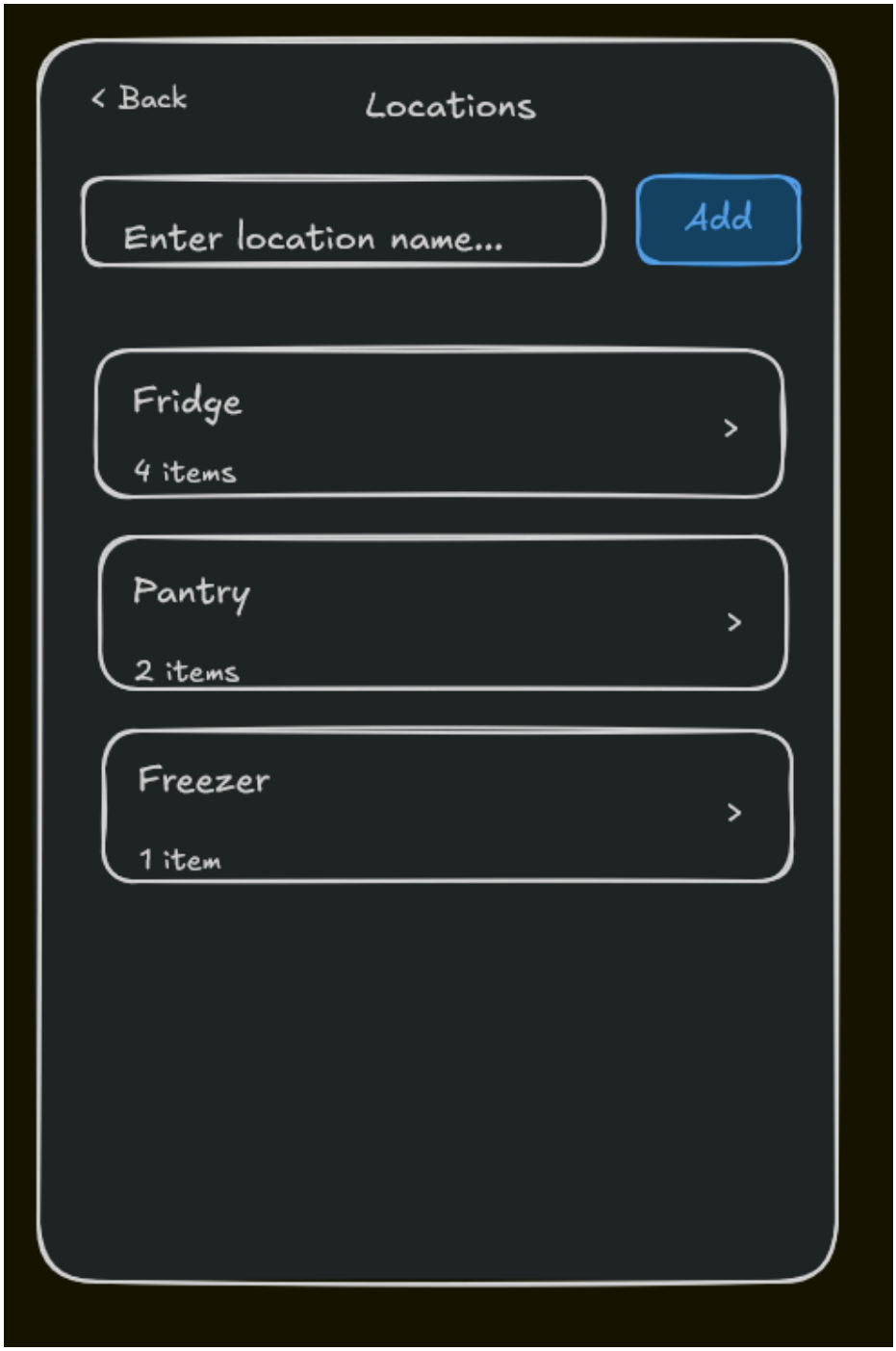
Shopping List:

Profile management:

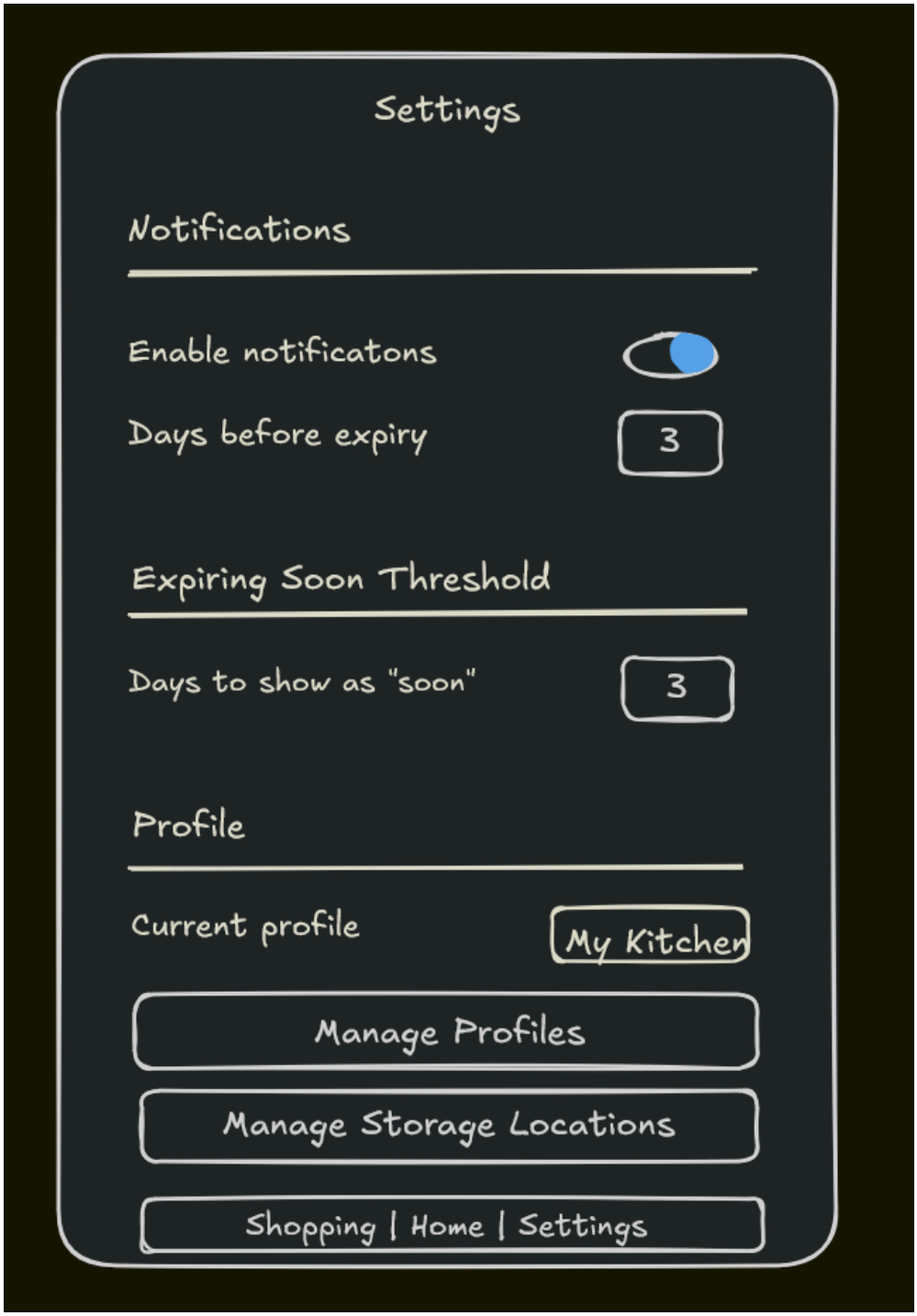
Expiring Soon:



Storage locations:



Settings:


Settings
Notifications
Enable notificatons
Days before expiry
3
Expiring Soon Threshold
Days to show as "soon"
3
Profile
Current profile
My Kitchen
Manage Profiles
Manage Storage Locations
Shopping | Home | Settings

Screen

Home/Dashboard

The first thing the user sees when opening the app. It displays their tracked items grouped by storage location. There will be an expiration banner at the top that shows a count of expiring/expired items and allows users to navigate to the Expiring Soon view. Below that, there will be a filter bar option that allows filtering by category or storage location. The floating action button in the right hand corner gives two options after tapping: go to the add items screen, or to the add storage locations screen. The bottom tab bar provides navigation to the Shopping List, Settings, or back to the Homepage. Furthermore, the locations will be collapsable for easier navigation.

Add Item

When the user clicks on the plus icon, they will be given an option to go to the add items screen. The user can select from a searchable predefined list or type a custom name. If the user selects an item from the predefined list, the expiration date will be auto filled based on the item's value in the default expiry lookup table. If the item is not in the predefined list and the user doesn't want to enter a date manually, the "use default" checkbox applies the 3 week fallback expiration date. Storage location is required. Optional fields include category, note, and photo.

Item Detail / Edit

Selecting a food item from the Homepage will take the user to the Item Detail page. It shows all of the details for a tracked item. The user can mark the item as used (they successfully used the item, did not waste it), mark it as discarded (had to throw it away due to spoilage or another reason), or delete it (they simply did not wish to track it anymore). The Edit button in the header switches the display to editable fields, allowing the user to change the category, expiration date, or add additional notes.

Shopping List

Selecting the Shopping List tab from the Homepage takes the user to an editable list of items that they plan to purchase. Users have the option to add new items to their shopping list, or edit items currently in it. Each item can be removed or moved to active tracking (which leads to a pre-filled Add Item screen). If a newly added item name matches an existing tracked item, a duplicate notice appears below the input.

Expiring Soon

The Expiring Soon page takes users to a list of all the items approaching or past their expiration date, sorted with the most urgent first. Each row shows the item name, storage location, and a color-coded label indicating days remaining or days past expiration.

Storage Locations

The Storage Location page shows each location with its active item count. Tapping a location shows the items stored there. Deleting a location that contains items triggers the confirmation dialogue. There will be the option for users to create new locations, rename existing ones, and delete empty ones.

Settings

The Settings page contains the notification toggle, the days before expiration threshold for notifications, the expiring soon display threshold, and a link to Profile Management.

Profile Management

This page is within the Setting page and displays a list of all local profiles with the active one highlighted. Users can create, rename, switch, or delete profiles. The system prevents deletion of the last remaining profile.

Key Decision Log

Shopping list duplicate detection

Decision: When the user adds an entry to the shopping list whose name matches an item already in active tracking, a notice will pop up but the system will not block the action.
Why: A non blocking notice respects user autonomy, while still providing useful information. The user might be aware that they already have an item at home, but may want more anyway.
Impact: The shoppingListRepositiory makes a query to match names against active tracked items whenever a new item is added to the shopping list.

Allowing multiple items of the same name

Decision: The system will not enforce item name uniqueness.
Why: Users may have multiple items of the same name with different expiry dates. This may be due to the nature of the product, their storage method which could impact shelf life, or maybe they bought the same item on different dates. Furthermore, they might have multiple of the item and want to store it in differnet locations.
Impact: The app will assign each entry a primary key instead of tracking food by names. Duplicate items will be expected and handled.

Photos stored as file paths

Decision: Photos are saved to the device file system via expo-file-system and referenced by file path in the database.

Why: Storing images directly would increase the size of the database and possibly affect query performance. A file system storage approach would solve this concern.
Impact: Photo deletion will have to be handled explicitly when a profile or item is deleted.

Local notifications over a background/polling service

Decision: Notifications are scheduled at the moment an item is created or edited using expo-notifications rather than using some background service that occasionally makes checks.
Why: Scheduled notifications are more efficient and is enough for the purposes of the application, the moment an item is created and the expiry date is specified the countdown to spoilage is deterministic.
Impact: The notificationScheduler schedules specific notification times with the mobile device's operating system via expo-notifications. It must also cancel old notifications and schedule a new one if the user makes an update.

React Native with Expo over Android-native Java

Decision: React Native + Expo will be the mobile framework.
Why: Although the team has more experience with Java and the frameworks that it offers, we are still limited in terms of mobile development experience. That means we would still have to learn Android studio, XML, and everything else that comes with android development regardless. Even though we have less experience with Javascript and React native, since both routes will require learning, we decided to take advantage of the React Native and Expo ecosystem and what they have to offer in terms of development.
Impact: The team will need to invest some time getting comfortable with JS, React native and Expo before development takes place.